Further Simplification of the K215 Correctness Proofs

Collection before quotient consequences,
shared register macros, and one AX effect proof

Research report and checked prototype packet

October 21, 2026

Abstract

The next useful simplification is to change which results are proved once. For Lean, transport the complete formula-indexed collection schema into the bisimulation quotient first, and derive Separation and Replacement there using ordinary equality. For Rocq, share unchanged register-machine macro proofs through a checked finite program slice. A third opportunity is to share the checked AX word interpreter and its effect proof: the actual 215 and 217 AX words differ semantically by one adjacent commuting pair.

These recommendations are supported by newly compiled components against the pinned repository. The Lean quotient consequences reproduce the exact existing public theorem types. The Rocq slice theorem is bound to the literal three program DAGs and independently checked with `coqchk`. The AX comparison proves equality of the complete maintained abstract effects for every register record. This packet is an additive proposal; it does not report a newly rebuilt machine-to-ZFC endpoint or a measured net production-source reduction.

1 Findings and recommended order

My first recommendation is **collection before quotient consequences**. The current Lean development already proves that the quotient satisfies collection for every actual first-order formula, including native equality. That result appears after Separation and Power in the import graph, although its proof needs neither axiom. Moving the environment-selection helper removes the accidental dependency. Separation and Replacement can then be consequences of one general theorem about functional images.

My second recommendation is **transport of unchanged register macros**. The Rocq macro libraries for the 215, 217, and 218 programs have identical 759-line bodies after identifier renaming. Their 53 literal macro entries refer, transitively, to only 89 program nodes, identical in all three DAGs. A small general theorem preserves every structured execution over this closed support.

The third recommendation is **one abstract AX effect proof**. The current Perm and Dom implementations each use a checked 156-atom word. Erasing routing labels reveals a common six-atom prefix, an exchanged pair `cons`/`readp1`, and a common 148-atom suffix. The exchanged operations commute on the actual register record. The resulting complete effect equality has been checked against both maintained implementations.

Proposal	What it shares	Evidence delivered
Collection first	One raw-to-quotient schema transport; native functional-image reasoning	Actual-source Lean adapters and axiom reports
Closed macro slice	One library of unchanged register-macro contracts	89-row checked binding; all-exit transport; recursive `coqchk`
Shared AX effects	Word interpreter, invariant induction, and symbolic effect proof	Actual `f236` equality for all abstract states

These are proof-organization changes. The mathematical collection implication is standard, and the repository already discusses related arguments. The new work here is the exact application to the current dependency structure, the finite macro support, and the checked equality of the two actual AX effects. Historical simplifications in the repository are credited separately.

2 Reviewed snapshot and current baseline

All repository claims in this report refer to

`6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95`.

The source was obtained through the GitHub connection and an isolated local checkout. No production source was edited.

The current published Lean checkpoint is phase fourteen. Its recorded endpoint closure contains 178 project modules and 2,093,112 source bytes. The independent source census in this packet reproduces those numbers and counts 40,517 physical lines. It follows imports from `BusyBeaver.ZFC.KEndpoint215` across the BusyBeaver, FirstOrder, ZF, and BoundedZFCConsistency source roots; the only remaining external import root is `Init`. A source census does not itself verify compilation. The complete endpoint evidence remains the repository's phase-fourteen record.[2, 3]

The existing Lean theorem is

$$\text{BlankHalts}(\texttt{machine215}) \longleftrightarrow \text{BProv}(\texttt{ZFCax_s}, [], \bot),$$

with the nonhalting equivalence obtained by negation. The recorded Rocq endpoint is `K215F.CorrectDense215.tm215_halts_iff_ZFC_inconsistent`. Its maintained build uses Rocq 9.1.0 and rocq-stdlib 9.0.0; the repository records a 237-source compilation, closed endpoint assumption reports, and recursive kernel checking.[1, 4]

Several useful ideas are already implemented or separately proposed:

- Shared derived rules and transitive-container contracts are in the phase-fourteen Lean checkpoint. The reported 12,116-byte reduction and 178-module closure belong to that work, not to this packet.
- The current Lean source correspondence uses normalized certificate machinery, and earlier phases already separated semantic interfaces from unrelated program proofs.
- The `quotient-unification` proposal already makes the CP/GIC quotient an instance of the `QHyp` construction. Its reported $+147/-748$ patch and 25-module check are prior work.

- The existing `shared-invariants/decoder-residual` component already covers all 4,751 source leaves and 6,176 guarded outcomes, beyond the earlier two-outcome pilot. It remains separately distinguished from a full endpoint adoption.
- The Rocq proposal queue already contains normalized and balanced source checkers and a checked 367-vertex affine-core route.

The proposal index and the individual receipts should be read together: some index introductions preserve earlier checkpoint descriptions. This report uses the exact source and the phase-fourteen record for the current maintained baseline.[9, 10, 11]

3 Collection before Separation and Replacement

3.1 The dependency that can be removed

The direct import in `KQ1Collect.lean` is `KQ1Power`. The mathematical ingredients used by its collection proof are: the quotient model; satisfaction of translated formulas; two-argument bisimulation invariance; and a representative for each entry of a supplied quotient environment. The last ingredient is currently exported by `KQ1Separation.liftEnv` and `mk_liftEnv`.

Consequently the present source order is effectively

$$\text{quotient model} \longrightarrow \text{Separation} \longrightarrow \text{Power} \longrightarrow \text{Collection}.$$

The useful order is

$$\text{quotient model and environments} \longrightarrow \text{Collection} \longrightarrow \text{native consequences}.$$

The native consequences include Separation and Replacement. This does not assume that arbitrary semantic predicates are definable. The transported theorem still quantifies over actual formulas and parameter environments.[5, 6, 7]

The supplied `QuotientCollectionFirst.lean` demonstrates the import cut: its only direct import is `KQ1Model`. It relocates the representative helper and retains the existing collection argument under a fresh namespace. That source has compiled successfully. Thus the absence of a Separation or Power dependency is supported by an actual build.

3.2 The exact functional-image lemma

Let V be a type and $E : V \times V \to \mathsf{Prop}$ any binary relation. No set-theoretic axioms are required for the following argument. For a relation r, the repository's collection witness has the form

$$\begin{aligned} \mathrm{IC}_E(r,a,b) \quad \Longleftrightarrow \quad & \left[\forall y \, (E(y,b) \Rightarrow \exists x \, r(x,y))\right] \\ & \wedge \left[\forall x \, (E(x,a) \Rightarrow (\exists y \, r(x,y)) \Rightarrow \exists y \, (E(y,b) \wedge r(x,y)))\right]. \end{aligned}$$

The first conjunct gives *global* support: it does not constrain the supporting input x to belong to a. Write $\mathrm{Collect}_E(r)$ for $\forall a \, \exists b \, \mathrm{IC}_E(r,a,b)$.[8]

Theorem 1 (Exact image from one guarded collection instance)**.** *Fix $a \in V$ and $r : V \times V \to \mathsf{Prop}$. Assume*

$$\forall x, y, z, \quad E(x, a) \wedge r(x, y) \wedge r(x, z) \;\Rightarrow\; y = z,$$

and assume collection for the single guarded relation

$$r_a(x, y) := E(x, a) \wedge r(x, y).$$

Then

$$\exists b \, \forall y, \qquad E(y, b) \;\longleftrightarrow\; \exists x \, \big(E(x, a) \wedge r(x, y)\big).$$

Proof. Apply the supplied collection instance to a, obtaining b, support, and coverage. If $E(y, b)$, support supplies an x satisfying $r_a(x, y)$, which is exactly the desired right-hand side.

Conversely, suppose $E(x, a)$ and $r(x, y)$. Coverage supplies some z with $E(z, b)$ and $r_a(x, z)$. Functionality on a implies $z = y$. Substitution gives $E(y, b)$. □

The relation may be partial. The proof uses neither totality nor a simultaneous choice of witnesses. It does not require Extensionality, Foundation, membership congruence, or an equivalence relation on V. The standalone Lean and Coq versions in this packet have no axioms.

Corollary 2 (Separation)**.** *Assume collection for*

$$E(x, a) \wedge \big(x = y \wedge P(x)\big).$$

Then there is b with

$$E(y, b) \longleftrightarrow E(y, a) \wedge P(y).$$

The proof applies the theorem to the partial identity on P. Its functionality follows from equality alone. The same observation explains why a *partial functional exact-image* Replacement interface can supply Separation. This depends on the precise Replacement interface; it should not be substituted without checking the premises of another formulation.

3.3 Why the guard is necessary

Omitting $E(x, a)$ from the relation is unsound for this collection interface. A four-element example makes the failure explicit. Take objects $0, 1, 2, 3$, whose membership extensions are respectively

$$\varnothing, \quad \varnothing, \quad \{0\}, \quad \{0, 1\},$$

and take $r = \{(0, 1), (1, 0)\}$. The relation is functional. Unguarded collection for r holds for every domain object: use an empty object for the empty domains and object 3 for the nonempty ones. Its global support permits both outputs.

For the domain object 2, however, the exact image is $\{1\}$. No object has that membership extension. Thus functionality and unguarded collection do not imply the exact-image conclusion. The included `guard_counterexample.json` records the finite witnesses; `scripts/inventory.py` reproduces the check.

This example also explains why replacing formula-indexed collection with an unrestricted semantic collection hypothesis would conceal a real obligation. The proposed Lean adapter constructs the guarded formula explicitly.

3.4 Formula indexing and variable order

The actual repository adapter is `GICConsequences.lean`. It defines

$$\mathrm{FullGIC}(E) := \forall \phi, \rho, \quad \mathrm{Sat}(E, \rho, \mathrm{gicForm}(\phi)).$$

There is no `EqualityFree phi` premise at this stage: equality has already been interpreted in the quotient.

The GIC matrix uses input at slot 0, output at slot 1. The repository's `relOf` convention uses output at slot 0, input at slot 1. To insert the domain as a parameter, define

old slot	0	1	$k+2$
new slot	1	0	$k+3$

and set

$$\mathrm{imageMatrix}(\phi) = \mathrm{Mem}(0,2) \wedge \mathrm{rename}(\mathrm{imageRename}, \phi).$$

At environment $[x, y, a, \rho_0, \rho_1, \ldots]$, this expresses $E(x,a) \wedge \mathrm{relOf}(E, \phi, \rho)(y, x)$.

For Separation, the partial-identity matrix is

$$\mathrm{filterMatrix}(\phi) = \mathrm{Eq}(0,1) \wedge \mathrm{rename}(\mathrm{succ}, \phi).$$

Its interpretation under $[y, x, \rho_0, \ldots]$ is $y = x \wedge \mathrm{Sat}(E, [x, \rho_0, \ldots], \phi)$. Both satisfaction calculations were checked using the actual `Form`, `Sat`, `relOf`, and renaming lemmas. These calculations bind the native argument to the repository's first-order syntax.

The native Replacement proof applies the functional-image theorem to `image_instance`. The native Separation proof specializes Replacement to the partial identity. Finally, `QuotientConsequenceAdapters.lean` reproduces the exact existing types of `quotient_separation` and `quotient_replacement`; each proof applies the native theorem to `quotient_fullGIC`.

3.5 Integration without a circular premise

One weaker route must survive the reorganization. `KQ1Power.trace_instance` calls `KQ1Separation.separation_instance` under only `ModelsGIC mem`, at the matrix `fMem 0 1`. The existing CP/GIC-to-`QHyp` adapter needs this raw trace construction to establish its power field. Replacing that weak premise by the already constructed quotient would risk requiring `QHyp` to build itself.

Move that raw trace machinery to a small upstream module, or retain the necessary compatibility declarations. The full quotient schema proofs can still use the new order. Also preserve the additional `SubD` conclusion of the old interpreted-separation lemma wherever a client needs it; native Separation alone does not assert that extra raw property.

The existing `quotient-unification` patch addresses a different duplication: two copies of the quotient construction. The present proposal changes the order of theorems *inside* the shared construction. They compose, but their source-size savings should not be added before an actual combined patch and its clients have been checked.[10]

3.6 A second checked use in the well-founded model

The generic module also exports `separation_of_replacement` independently of GIC. It takes the repository's partial functional exact-image Replacement interface and derives Separation by the partial

identity. `WFSeparationAdapter.lean` applies it to the existing `WFModel.domain_replacement`, reproducing the exact public type of `WFModel.domain_separation`. The proof body changes from 14 lines to two. The adapter's axiom report is `propext` and `Quot.sound`.

The implementation order is sound: the current `WFModelCore.domain_replacement` does not use `domain_separation`, so Separation can be moved below Replacement. This pattern cannot be applied indiscriminately. In `ConstructibleZF`, `inL_repl` explicitly invokes `inL_sep`; deriving that Separation theorem from Replacement would create a cycle. The well-founded adapter is checked; no such constructible-model replacement is proposed.

3.7 Evidence and honest size accounting

The final collection component checks 35 modules with Lean 4.32.0, including five proposed modules and their actual repository dependencies. The standalone functional-image theorem has no axioms. The native formula-indexed Separation and Replacement theorems use only `propext`; the quotient adapters use the same standard trio as the existing quotient construction: `propext`, `Classical.choice`, and `Quot.sound`.

The current Separation, Replacement, and Collection files have 118, 143, and 92 physical lines, respectively: 353 in total. The two public quotient proof bodies contain $14+25 = 39$ lines and become $2+2 = 4$ lines. That is a local 35-line body reduction. The generic consequence module adds 110 lines, while environment helpers move and raw compatibility must be retained. Therefore 35 is not a net repository saving. The important structural gain is one quotient schema transport, and unused specialized helper developments become candidates for deletion after a reverse-use audit. The additional well-founded reuse saves another 12 local proof-body lines. A conservative uncommitted layout budget removes 71 lines of unused raw Replacement helpers, 16 lines of the interpreted Separation helper, and those 47 body lines, while adding the 110-line generic module. That leaves about 24 lines before changed imports and compatibility wrappers. It is a design budget, not an adopted source diff; `integration-budget.json` records the assumptions.

4 Rocq: one macro library over a checked program slice

4.1 The duplication and its exact support

At the reviewed snapshot, the maintained Rocq files have the following sizes:

File	Lines	Bytes
`215/ProgLibPerm.v`	780	37,951
`217/ProgLibDom.v`	781	38,002
`218/ProgLib219.v`	779	37,829

Starting at the first `From Coq Require Import`, and renaming only namespace and DAG identifiers, all three bodies are byte-identical: 759 lines and 36,402 bytes.

A literal extraction of the `sem (ENode n)` entries yields 53 roots. Closing them under both child-node edges yields 89 nodes. Every row in that support agrees in the actual `dagPerm`, `dagDom`, and `dag219`. The census script, hashes, root list, and complete support are included.[14]

The changed dispatcher and AX nodes are outside this support. For example, nodes $139, 141, 147, 236$, and the new nodes 467–486, cannot be treated as part of a universal shared program. Node 141 is especially easy to overgeneralize: it occurs in the atom catalogue but belongs to the dispatcher, outside the shared macro slice.

4.2 The transport theorem

For a finite node set S, let $\mathrm{Allowed}(S, e)$ mean that e is primitive or $e = \mathrm{ENode}(n)$ with $n \in S$. Define

$$\begin{aligned} \mathrm{Closed}(p, S) &:\Longleftrightarrow \forall n \in S, \quad \mathrm{Allowed}(S, \mathrm{car}_p(n)) \wedge \mathrm{Allowed}(S, \mathrm{cdr}_p(n)), \\ \mathrm{Agrees}(p, q, S) &:\Longleftrightarrow \forall n \in S, \quad \mathrm{nodes}_p(n) = \mathrm{nodes}_q(n). \end{aligned}$$

Theorem 3 (Exact structured-run transport)**.** *If* $\mathrm{Closed}(p, S)$ *and* $\mathrm{Agrees}(p, q, S)$*, then for any allowed edge* e*, any register files* R, R'*, and any exit* x*,*

$$\mathrm{Run}_p(e, R, x, R') \longleftrightarrow \mathrm{Run}_q(e, R, x, R').$$

Proof. Induct on a structured run. Each primitive constructor ignores the program. For a node constructor, agreement rewrites the two selected child edges, and closure supplies the allowed-edge hypotheses for the induction steps. The `run_node_cdr_continue1` constructor has a third premise, restarting the same node after the car and cdr runs. That node is still in S, so the third induction hypothesis applies directly.

This proves forward transport. Closure of q follows by rewriting its rows using agreement; reverse transport applies the same induction with the programs exchanged. □

The result preserves normal, break, continue, and halt exits exactly. It quantifies over arbitrary natural-valued register files. It does not rely on a register bound, whole-program equality, or a termination argument for loops.

4.3 A finite checker bound to the literal programs

`RMSlice.v` proves soundness of a Boolean checker that tests closure and row agreement on a supplied list. The list is untrusted input. The concrete module `K215MacroSlice.v` proves the finite checks and exports:

```
macro_run_215_217
macro_run_215_218
```

Their conclusions are the exact `RM.run` equivalences at all 53 roots. The generic and concrete new files total 161 source lines.

All six modules, including the four unchanged dependency sources, compile with Coq 8.18.0. Three assumption reports are closed. Recursive `coqchk -silent K215MacroSlice` succeeds. The dependency manifest verifies the included source snapshots against the pinned repository.

This is a source-semantics result. It does not assert equivalence of the complete 215 and 217 programs, their command decoders, or their binary Turing machines.

4.4 How to use it in the maintained development

First extract the shared register record and the structural sequence rules with an explicit program parameter. Keep one set of named macro contracts for a reference program. For each other candidate, prove the finite slice check and transfer the macro contracts.

Preserve target-specific expansion rules and the dispatcher, AX, and root proofs. Update `regenerate 215.py` and `regenerate217.py` with the source reorganization; otherwise regeneration would restore the copied libraries.

The measured duplication makes this an attractive maintenance change. It does not justify claiming that all 759 lines can be deleted twice without replacement: shared definitions and compatibility wrappers remain. Nor does it establish a smaller single-endpoint import closure or faster kernel checking. Those are separate measurements after integration.

5 Lean: one AX word interpreter and one effect proof

5.1 What is already present

The recent conversion of the AX routine to a checked macro word is a major existing simplification. The current `L2AxiomsPerm` and `L2AxiomsDom` each contain 161 lines, with 7,563 and 7,536 bytes. They repeat the atom type, atom effects, run and cleanliness contracts, word checker, evaluator, and composition inductions.[15, 16]

The prototype generalizes that architecture rather than returning to per-node proof generation. `GenericMacroWord.lean` takes a program, an abstract state and encoding, an invariant, atom edges and effects, a terminal gate, and the macro contracts. One induction proves invariant preservation; one checked-word induction proves the resulting `RMDag.Run` contract.

Each candidate must retain its own literal word and its own equation binding that word to its actual program. Sharing semantic evaluation does not identify the two programs' control-flow labels.

5.2 The complete literal effect equality

Both actual words contain 156 macro atoms. They have:

$$\underbrace{\text{common prefix}}_{\text{6 atoms}} ; \begin{cases} \texttt{cons};\texttt{readp1} & \text{Perm}, \\ \texttt{readp1};\texttt{cons} & \text{Dom}, \end{cases} \underbrace{\text{common suffix}}_{\text{148 atoms}}.$$

This comparison is made after erasing node-routing labels. The labels at positions 5 and 6 differ and remain part of the separate program-binding checks.

On the actual abstract state record, `readp1` operates on `p1, pl`, while `cons` operates on `tw, ws`. Their total effects therefore commute:

$$\text{cons}(\text{readp1}(s)) = \text{readp1}(\text{cons}(s)).$$

The concrete Lean proof is record case analysis followed by reflexivity. There is no cleanliness premise on this equality.

The generic `eval_swap` theorem places this equality under arbitrary prefixes and suffixes; `eval_labels` removes irrelevant routing labels. The supplied `AXWordsEqual.lean` then proves, using the actual

maintained definitions,

$$\forall s : \texttt{L2K.AS}, \qquad \texttt{L2Perm.f236}(s) = \texttt{L2Dom.f236}(s).$$

The axiom report for this theorem contains only `propext`.

Consequently the exact existing Dom closed-form AX specification follows from the Perm theorem with the two-line proof:

```
rw [<- f236_eq]  -- ASCII rendering of Lean's left arrow
exact L2Perm.ax_eval np t S c a h pl
```

The source file contains the exact Lean syntax. This derived specification inherits the existing standard trio of axioms. The effect equality does not remove `Clean` from the run contracts: the total abstract transformer and its concrete execution theorem have different hypotheses.

5.3 Integration and scope

Move the common atom signature and effects into one shared module, together with the invariant and word soundness theorem. Each program keeps a literal word, the checked binding to its DAG, and wrappers for the existing `f236`, `l236`, `c236`, and `ax_eval` names.

The generic interpreter has compiled, and the complete actual effect equality has compiled. The current production `l236` proofs have not been replaced in this packet. A 33-module check covers 30 existing repository dependencies and three proposed modules. No machine endpoint is rebuilt.

The same word architecture could simplify the roughly 193 KB generated `L2Axioms222` and `L2Axioms219` developments. Those files are outside the current K215 endpoint closure, so their potential savings belong to repository maintenance, not to a smaller K215 proof closure.

6 Preserving the different semantic and syntactic proof routes

The two languages currently realize the set-theoretic lower chain differently. Lean constructs a host-language quotient type, where equality is ordinary Lean equality. The pinned Rocq upstream develops object-language natural-deduction derivations such as

$$G \vdash_C \exists c\ \mathrm{qimage}(t, a, c, \phi, \sigma).$$

The files `KQuotReplacement.v` and `KQuotSeparation.v` use `qtrans`, equality modulo `deqv`, explicit formula bounds, schema membership, and substitution lemmas. The upstream `KCollection.doma in_collection` supplies both coverage and support, so the same mathematical image argument is available.[12, 13]

A production Rocq version must package that argument as a *formula-level derivation*. Its inputs should include supported collection, functionality modulo `deqv`, congruence of the translated relation, domain and marker guards, and the boundedness/substitution facts that justify the selected axiom instances. The output should preserve the existing `qimage` or `qseparation` formula.

The standalone `CollectionFirst.v` included here verifies the small host-`Prop` argument constructively and passes `coqchk`. It is a useful shared specification; it does not by itself replace those syntactic Rocq proofs. Conflating these levels would leave the required calibration argument unfinished.

7 Adoption plan and remaining opportunities

A practical adoption order is:

1. **Integrate the Lean collection consequence module.** Move environment selection upstream, make `KQ1Collect` independent of Separation and Power, and replace the two quotient schema bodies. Retain the weak raw trace route and all required old signatures.
2. **Integrate the Rocq macro slice.** Separate program-independent structural helpers from actual-DAG expansion, share the macro library, and update both generators. The finite support checks should stay close to the program data.
3. **Integrate the AX sharing.** Use one interpreter and one abstract effect derivation, preserving separate literal program checks. Keep public root-module interfaces unchanged.
4. **Reconcile already checked certificate proposals.** Choose the intended Rocq source correspondence implementation and bind it to its producer, manifest, and full endpoint. Integrate the existing complete semantic decoder/residual component through the intended endpoint wrapper.

For each adoption, rebuild the affected dependency and reverse-client closure, then the exact endpoint and its assumption reports. Compatibility wrappers matter here: several older candidates and model interfaces are maintained alongside K215. A passing isolated component is valuable evidence but does not certify the final import graph after source replacement.

The next substantial runtime-source reduction would replace occurrence-specific suffix checks by parametric residual contracts. Those contracts must quantify over the accepted `RolePath` language and preserve the marker, carry, and continuation conditions used by the suffix. A census of 97 short contexts is not a proof of 97 universal contracts. The existing complete residual checker can remain a fallback for cases not yet handled parametrically. Its integration alone adds an interface and does not demonstrate a source-size reduction.[11]

8 Validation ledger and quantitative claims

Component	Checked scope	Result
Lean collection route	35 modules; actual first-order syntax, exact quotient schema signatures, and the well-founded model adapter	Pass, Lean 4.32.0
Coq semantic core	Functional exact image and Separation; recursive kernel recheck	Closed, Coq 8.18.0
Rocq macro slice	Six modules; three literal DAGs, 53 roots, 89 rows; recursive kernel recheck	Closed, Coq 8.18.0
Lean AX sharing	33 modules; generic word soundness and actual complete effect equality	Pass, Lean 4.32.0
Current Lean inventory	178 project modules; 2,093,112 bytes; 40,517 physical lines	Source audit only

The source inventory is independently reproducible. The baseline complete endpoint checks are credited to the repository's records. This packet does not claim a fresh 178-module K215 build or a fresh Rocq 9.1 endpoint check. Coq 8.18.0 was used for the portable Rocq-compatible prototypes; production adoption should run the repository's pinned Rocq toolchain.

The reported line counts are physical source lines including comments. Byte counts are UTF-8 file sizes. The macro duplication count additionally specifies its exact normalization. No statement here equates source-line reduction with smaller proof terms, fewer kernel reductions, or shorter build time. Those quantities should be measured separately after adoption.

The successful new proofs use no `sorry`, `Admitted`, or added logical axiom. The retained initial AX-word attempt failed only because the concrete comparison exceeded the default recursion limit. The accepted proof uses `maxRecDepth 4096`, matching the maintained AX modules. Existing dependency linter warnings are visible in the collection build log; they are not hidden or reclassified as new proof obligations.

9 Artifact map and reproduction

The archive contains this article as LaTeX and compiled PDF, the new proof sources, portable build scripts, source-bound receipts, finite audits, checksums, and inherited attribution notices. It intentionally omits compiled `.olean`/`.vo` objects and downloaded compiler distributions.

Path	Purpose
`code/lean/collection-first/`	Five Lean modules, source-level schema build, evidence
`code/lean/macro-word/`	Generic interpreter, actual commutation, complete AX effect equality
`code/rocq/macro-slice/`	Generic transport, literal support binding, pinned dependencies
`code/rocq/collection-first/`	Constructive mathematical core in Coq
`scripts/inventory.py`	Exact endpoint census and guard counterexample
`evidence/`	Scope summary, inventory, and finite counterexample
`MANIFEST.sha256`	Hashes of delivered files

Use a checkout at the pinned commit. The Lean source roots required by the scripts are those listed in the inventory section; the scripts do not modify the checkout. Representative commands are:

```
python3 code/lean/collection-first/check_repository.py \
  /path/to/ProveIt /path/to/lean-4.32.0/bin/lean

python3 code/lean/macro-word/build_prototype.py \
  --repo /path/to/ProveIt --lean /path/to/lean-4.32.0/bin/lean \
  --out /path/to/new-build

python3 code/rocq/macro-slice/build.py

python3 scripts/inventory.py \
  --repo /path/to/ProveIt --output /path/to/new-audit
```

The Rocq slice driver uses `COQC` and `COQCHK` when supplied, otherwise the tools on `PATH`. The collection-core driver is similar. The macro-word build checks source hashes against its shipped receipt. The inventory script verifies the exact commit and expected closure totals.

The checked components are intended as concrete material for the next reviewable repository changes. The adoption decision should be based on the combined source diff, preserved endpoint types, producer synchronization, and the final assumption reports.

References

[1] ProveIt, *K215 candidate and checked-proof overview*, pinned review commit. https://github.com/VladimirReshetnikov/ProveIt/blob/6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95/Computability/BusyBeaver/Research/ZFC/candidate-k215/README.md

[2] ProveIt, *Fourteenth K215 proof-simplification checkpoint*. https://github.com/VladimirReshetnikov/ProveIt/blob/6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/phase14/README.md

[3] ProveIt, `KEndpoint215.lean`. https://github.com/VladimirReshetnikov/ProveIt/blob/6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KEndpoint215.lean

[4] ProveIt, *Rocq endpoints and the 215 build route*. https://github.com/VladimirReshetnikov/ProveIt/blob/6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95/Computability/BusyBeaver/Research/ZFC/k225-formal/rocq/rocq-extension/README.md

[5] ProveIt, `KQ1Collect.lean`. https://github.com/VladimirReshetnikov/ProveIt/blob/6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KQ1Collect.lean

[6] ProveIt, `KQ1Separation.lean`. https://github.com/VladimirReshetnikov/ProveIt/blob/6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KQ1Separation.lean

[7] ProveIt, `KQ1Replacement.lean`. https://github.com/VladimirReshetnikov/ProveIt/blob/6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KQ1Replacement.lean

[8] ProveIt, `ProfileCollection.lean`, definitions of collection support and coverage. https://github.com/VladimirReshetnikov/ProveIt/blob/6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/ProfileCollection.lean

[9] ProveIt, *Proof-simplification proposal index*. https://github.com/VladimirReshetnikov/ProveIt/blob/6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/proposals/README.md

[10] ProveIt, *One bisimulation quotient: CP/GIC through a QHyp adapter*. https://github.com/VladimirReshetnikov/ProveIt/blob/6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/proposals/quotient-unification/README.md

[11] ProveIt, *Decoder-prefix and residual coverage component*. https://github.com/VladimirReshetnikov/ProveIt/blob/6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/proposals/shared-invariants/decoder-residual/README.md

[12] Elliot Glazer et al., `k230-rocq`, `KQuotReplacement.v`, commit `3142e4dd980f0be12f4abb636c4ae2528afc17eb`. https://github.com/elliotglazer/k230-rocq/blob/3142e4dd980f0be12f4abb636c4ae2528afc17eb/coq/settheory/KQuotReplacement.v

[13] Elliot Glazer et al., `k230-rocq`, `KCollection.v`, same commit. https://github.com/elliotglazer/k230-rocq/blob/3142e4dd980f0be12f4abb636c4ae2528afc17eb/coq/settheory/KCollection.v

[14] ProveIt, `ProgLibPerm.v`, maintained K215 macro library. https://github.com/VladimirReshetnikov/ProveIt/blob/6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95/Computability/BusyBeaver/Research/ZFC/k225-formal/rocq/rocq-extension/coq/215/ProgLibPerm.v

[15] ProveIt, `L2AxiomsPerm.lean`, checked AX word for K215. https://github.com/VladimirReshetnikov/ProveIt/blob/6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/L2AxiomsPerm.lean

[16] ProveIt, `L2AxiomsDom.lean`, checked AX word for K217. https://github.com/VladimirReshetnikov/ProveIt/blob/6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/L2AxiomsDom.lean